

大三下期中專題報告

專題名稱：可離線運作之會議紀要系統

指導教授：吳齊人 教授

成員：B1229040 陳奕蓁

B1229042 楊程宇

B1229052 周姵妍

B1229057 陳泓瑜

1.1 規格目的

- 1.目的：紀錄會議過程並整理重點，以藍芽方式傳送給連接設備
- 2.設備：麥克風、resberrypi、聲音轉文字的本地模型、整理文字重點的本地模型
- 3.軟體規格：
 - a.接收聲音並轉為文字的模組
 - b.接收文字並整理文字重點的模組
 - c.將結果傳到連接裝置的藍芽模組
- 4.硬體規格：麥克風、存放模型及rasberrypi

1.2 規格範圍

- 1.時間：三十分鐘及以下
- 2.人數：五人以下，不超過兩個聲音同時說話
- 3.限制：建議每位與會成員在第一次發言時先表示自己的姓名

2.1 系統目標

- 1.自動化產出結構化重點，大幅節省人工整理紀錄的勞務成本
- 2.建立全離線運算環境，確保機密會議資訊達成物理級資安防護
- 3.發揮樹莓派體積優勢與整合封裝，打造隨處部署的行動紀錄器

2.2 系統範圍

1. 輸入：現場即時錄音
2. 核心技術：ASR 語音轉文字、SRT 解析、LLM prompt 生成與會者、重點整理、決策、待辦事項
3. 輸出：生成 txt 綜合報告，並透過藍牙進行點對點傳送

2.3 系統架構

1. 硬體驅動與音訊捕獲模組：透過 Subprocess 調度 arecord 與 ffmpeg 直接驅動硬體介面，實現音訊即時採集並封裝為 MKV 格式。
2. 離線 AI 推論引擎：整合 llama.cpp 框架與微調後的 Qwen 模型，於本地端完成語義分析與摘要，徹底免除外部 API 依賴。
3. 系統匯流排與狀態管理：結合 D-Bus 與 BlueZ 實現跨裝置藍牙通訊，並透過 JSON 快取機制維護分段處理過程中的數據一致性。

2.4 軟/硬體建構項目需求概述

需求	用處	備註
Raspberry Pi 5	主運算平台	16GB RAM
USB 麥克風	音訊輸入裝置	目前使用logitech視訊攝像頭
藍牙模組	結果傳輸介面	內建
FunASR	語音辨識套件	含 Paraformer-zh、FSMN-VAD、CT-Punc、CAM++ 模型
llama.cpp	本地 LLM 推論框架	
Qwen3	主要語言模型	4B-Instruct-Q8_0.gguf
OpenCC	繁簡中文轉換	
Python 3.10+	程式運行	Conda 虛擬環境

2.5 軟/硬體環境

- 作業系統：Raspberry Pi OS (64-bit , Debian Bookworm)
- 執行環境
 - asr_env：FunASR / 語音辨識 Conda 環境
 - pkd_env：llama.cpp / LLM 推論 Conda 環境
- 模型與資料路徑
 - 模型儲存路徑：/home/cgu-csie/
 - 模型快取：/home/cgu-csie/.cache/modelscope
 - 離線模式：HF_OFFLINE=1、TRANSFORMERS_OFFLINE=1 (不依賴外部網路)
- 系統資源配置
 - LLM 執行緒數：4
(OMP_NUM_THREADS=4)
 - Context window：2048 tokens (一般) / 8192 tokens (摘要模式，依 RAM 動態調整)
 - GPU 加速：n_gpu_layers=0 (純 CPU 推論)
 - 記憶體監控閾值：10 GB
(ModelConfig.MEMORY_CONFIG)

2.6 一般限制

1. 時間限制：30 分鐘以下。系統以每 10 分鐘為單位自動切片處理
2. 人員與音訊限制：5 人，同時發言人數不超過 2 人。首次發言自報姓名，以利人物識別
 - a. 說話者識別上限：50 人
3. 模型限制：
 - a. 最大 context window：8192 tokens (16GB RAM 版本)
 - b. 單段逐字稿輸入上限：1500 字元
 - c. LLM 推論為純 CPU 模式，不支援 GPU 加速

3.1 軟/硬體建構項目架構

1. 音訊擷取層：透過 arecord 與 ffmpeg 進行即時錄音與轉檔 (MKV)。RealtimeASR 類別 (speech/pipeline_mkv_read.py) 負責音訊分段與說話者聲紋識別
2. AI 推論層：llama.cpp 載入 Qwen 模型，由 extractors/ 下的五個模組 (PeopleExtractor、KeypointsExtractor、DecisionsExtractor、ActionsExtractor、SummaryGenerator) 依序分析。參數由 config/model_config.py 統一管理。
3. 輸出傳輸層：分析結果彙整為 txt 報告，pkd_worker.py 透過 BlueZ D-Bus 介面以藍牙點對點傳送連接裝置。

3.2 軟/硬體組件設計說明

1. Raspberry Pi 5 16GB：支撐多模型並行載入，並發揮其原生處理器性能
2. FunASR AutoModel：選用 Paraformer-zh（語音辨識）、FSMN-VAD（語音活動偉測）、CT-Punc（標點符號還原）、CAM++（說話者辨識）來支援更複雜的判斷
3. llama.cpp（Llama 類別）：針對 CPU 指令集優化，搭配 8-bit 量化 (Q8_0) 的 Qwen3 模型，在維持高精確度的前提下，確保於樹莓派端達成即時推論。
4. pkd_worker.py：調度 SRT 切片與資源排程，確保錄製與分析流程互不干擾
5. config/model_config.py：透過 ModelConfig 集中管理超參數，提升系統移植性
6. print_log_utils.py：建立具時間戳記的日誌，便於追蹤離線環境下的系統狀態

3.3 介面設計說明(本系統採用模組化 CLI 互動設計，將核心功能拆分為三個獨立執行單元)

1. 語音處理層 (run_asr_conda.py)：負責底層音訊驅動與逐字稿生成，支援串流與檔案雙模式。
2. 核心分析層 (run_pkd_conda.py)：負責說話者辨識與語意切片，為系統的運算中樞。
3. 摘要萃取層 (run_summary_conda.py)：利用語言模型針對分析結果進行二次加工，產出結構化報告。

3.4 作業程序設計說明

主控流程由 pkd_worker.py 協調，執行順序如下：

1. 啟動 RealtimeASR，進行即時錄音與語音辨識，每 10 分鐘自動切片輸出 SRT 檔
2. pkd_worker.py 監控輸出目錄，俾測新 SRT 後呼叫 SRTParser 解析並由 SRTSegmentizer 分段
3. 依序執行 5 個 Extractor，每段最多輸入 1500 字元至 LLM
4. SummaryGenerator 整合所有結果，以 JSON 格式輸出會議標題與摘要
5. 最終報告彙整為 txt 檔，透過 BlueZ D-Bus 藍牙介面傳送

3.5 輸入/輸出設計說明

類別	設計重點	說明內容範例
輸入設計	參數化控制	引入 overlap 緩衝機制（預設 5s）以解決即時處理時的語音斷句遺失問題。
輸出設計	多態產出	系統提供三層級輸出：1. 原始逐字稿 (SRT) 2. 機器可讀快取(JSON) 3. 人類決策摘要(TXT)。
資料整合	日誌追蹤	透過 print_log_utils 實現標準化執行紀錄，確保系統在高負載環境下具備可追蹤性。

3.6 其他設計說明

1. 資源監控與穩定性設計：系統即時監控記憶體並設定 10GB 閾值，一旦超標即觸發 gc.collect 進行主動清理。針對運算異常具備 3 次自動重試機制
2. 離線安全與本地環境設定：透過設定 HF_OFFLINE 等環境變數，強制系統進入全離線模式。模型統一存放於本地快取路徑，確保在無網路環境下仍能穩定執行推論
3. 動態聲紋辨識算法：採用 CAM++ 提取 192 維聲紋向量，以 0.7 相似度為判定閾值，並透過 $\alpha=0.2$ 的學習率動態更新聲紋原型。系統最高支援 50 位 說話者辨識

4 需求制設計之追溯與版本管理

1. 版本管理策略：採用 Git 分支策略管理開發流程，區分穩定版本與硬體整合分支，確保功能開發與 Raspberry Pi 測試並行且可追溯。
2. 模組化與組態管理：採高內聚設計，將各項分析功能封裝為獨立 Extractor；所有路徑與超參數由單一類別控管，提升維護效率。
3. 需求對應與追溯：建立模組與需求的對應架構：由語音、分析及藍牙傳輸模組嚴格對接開發需求，確保系統功能具備高度可驗證性。

附錄 - 檔案與程式式對照表

檔案名稱	對應程式模組	功能設計說明
speech/pipeline_mkv_read.py	RealtimeASR	即時音訊擷取、說話者辨識與 SRT 封裝。
speech/pipeline_mkv_write.py	RealtimeASR (寫入)	負責錄音旗標控制與音訊緩衝持久化。
speech/run_asr_conda.py	main()	ASR 流程的獨立啟動入口點。
extractors/base_extractor.py	BaseExtractor	定義 llama.cpp 推論的共用基底邏輯。
extractors/people_extractor.py	PeopleExtractor	執行與會人物辨識與發言角色歸納。
extractors/keypoints_extractor.py	KeypointsExtractor	提取會議對話中的核心關鍵要點。
extractors/decisions_extractor.py	DecisionsExtractor	識別並紀錄會議中達成的具體決策。
extractors/actions_extractor.py	ActionsExtractor	彙整會議產出的待辦行動與任務清單。
extractors/summary_generator.py	SummaryGenerator	生成會議標題與結構化全文摘要。
config/model_config.py	ModelConfig	統一配置模型路徑與推論超參數。
pkd_worker.py	PKDWorker	主控程序，同步協調 ASR 與 LLM 流程。
print_log_utils.py	setup_print_logging	全域日誌攔截與時間戳記日誌輸出。
core1.py	SRTParser	負責 SRT 檔案解析與語義區塊分段。
run_pkd_conda.py	main()	整合系統主流程的啟動入口。
run_summary_conda.py	main()	獨立執行摘要產出的啟動入口。

附錄 - 檔案與報表對照表

類別	檔案名稱	關鍵內容與設計用途	
最終報表	*_meeting_summary.txt	TXT	核心產出 ：整合標題、摘要、人物、重點與決策之完整報告。
輔助報表	output_*.srt	SRT	時序紀錄 ：提供含時間戳記之逐字稿，供使用者回溯會議細節。
中間資料	output_*_pkd_cache.json	JSON	分析快取 ：紀錄 PKD 處理之關鍵項目，支援斷點續傳與二次分析。
中間資料	output_*_summary_cache.json	JSON	摘要快取 ：儲存 LLM 生成之結構化摘要，確保資料一致性。
系統日誌	output_*_run.log	TXT	運行追蹤 ：詳列各模組執行時間與狀態，作為系統除錯與維護依據。
測試數據	project_test_*.xlsx	XLSX	品質驗證 ：儲存基準測試資料，用於評估系統識別之精確度。